

Operating Systems – General Principles

Sistemas de Computadores

Departamento de Engenharia Informática

Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Overview

- OS services
- Architectural support

Operating system

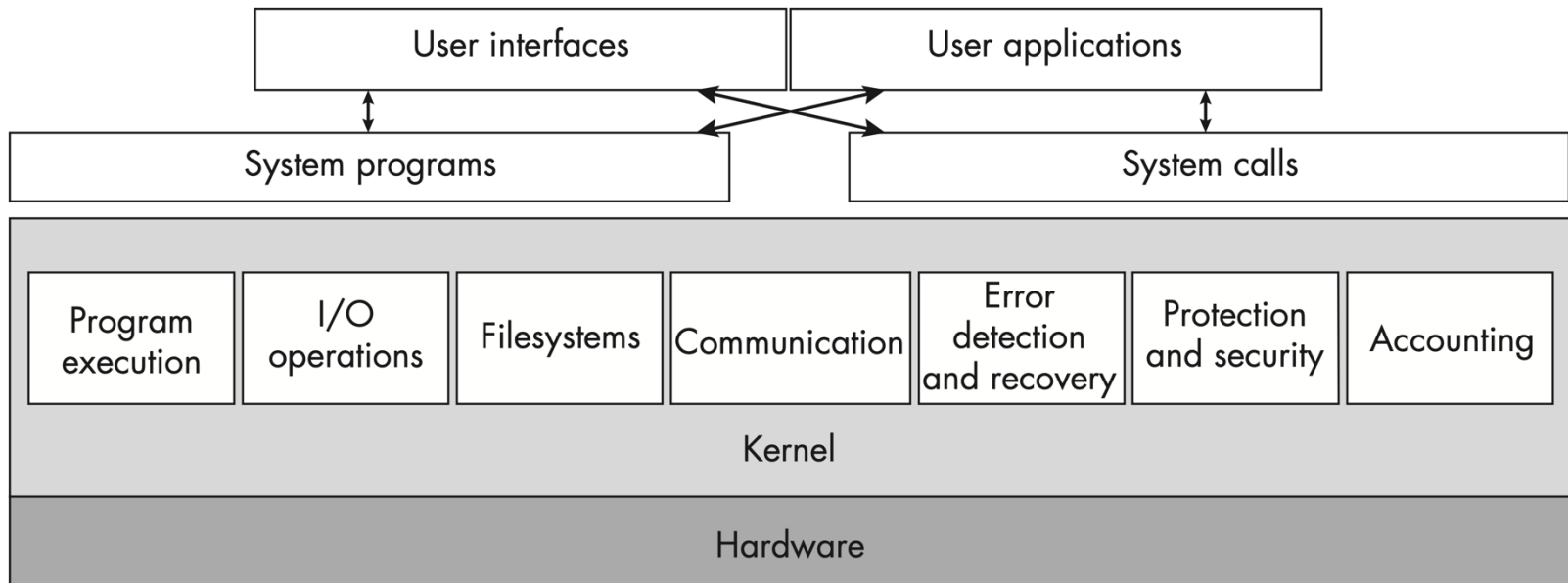
- **The operating system (OS) is the collection of all software that provides services to applications and users and manages and protects all hardware resources**
 - The OS provides abstractions such as processes, files, and virtual memory
- **The OS is a **resource manager****
 - Decide between competing requests to ensure an efficient and fair use of resources
- **The OS is a **control program****
 - Controls the execution of programs to prevent errors and inappropriate use of hardware

Why do we need operating systems?

- The OS simplifies computing by providing a structured environment for running applications efficiently
- OS provides **convenience**
 - High level of abstraction of physical resources
 - Allows the creation of more complex systems and code portability between different architectures
- OS enables **efficient sharing** of limited resources
 - Protection among applications

Operating system services

- Although the number and specific type of services differ between the various OSes, we can identify a common set



Operating system services

- **One set of services provides functions that are helpful to the user**
 - User interface, program execution, I/O operations, filesystem, communications (inter-process, networking), error detection
- **Another set of functions exists for ensuring the efficient operation of the system itself**
 - Resource allocation, accounting, security, protection
- **Key goals are to enforce **protection** and **resource sharing****
 - Particularly important when allowing the concurrent execution of multiple programs

Operating system design patterns

- **Operating systems rely on recurring design patterns to manage complexity, modularity, and reliability**
 - Depends mainly on how the various common components are interconnected and melded into the kernel
- **Understanding these patterns helps in:**
 - Designing robust concurrent systems
 - Avoiding common synchronization errors
 - Structuring inter-process communication correctly
- **A supporting document with additional details and examples is available on Moodle**

Overview

- OS services
- **Architectural support**

Architectural support

■ Why start our discussion with hardware?

- Hardware provides **mechanisms**
- The OS implements **policies** using those mechanisms
- When architectural support is well designed, applications can remain largely unaware of hardware details

■ A modern OS is deeply coupled with hardware features, relying on them to provide abstraction, protection, and performance

- Architectural support can greatly simplify or complicate OS tasks
- E.g., early systems without hardware support for virtual memory made memory management complex or infeasible

Types of architectural support

■ Manipulating privileged machine state

- Protected instructions – a subset of instructions of every CPU is restricted to be used only by the OS
- E.g., manipulate device registers, TLB entries, etc.

■ Generating and handling “events”

- System needs mechanisms for “exceptional control flow”
- E.g., interrupts, exceptions, system calls, etc.

■ Memory protection

- OS must be able to protect programs from each other
- OS must protect itself from user programs

Multiple processor operating modes

- Ensure that an incorrect (or malicious) program cannot cause other programs, or the OS itself, to execute incorrectly
- To do so, it must be able to distinguish between the execution of OS code and user-defined code
- Minimal requirement is to have a **kernel mode** and a **user mode**
 - The CPU physically enforces different constraints on programs operating in different modes

Multiple processor operating modes

- **The kernel resides at a higher level of privileges when compared to user or system applications, called **kernel space****
 - Includes protected memory space and full hardware access
 - Only the OS can:
 - Directly access I/O devices (disks, printers, etc.)
 - Manipulate memory management state
 - Manipulate protected control registers
 - ...
- **In contrast, applications run in **user space****
 - Level that imposes various restrictions on the operations that a process can perform
 - Observation: most instructions are perfectly safe!

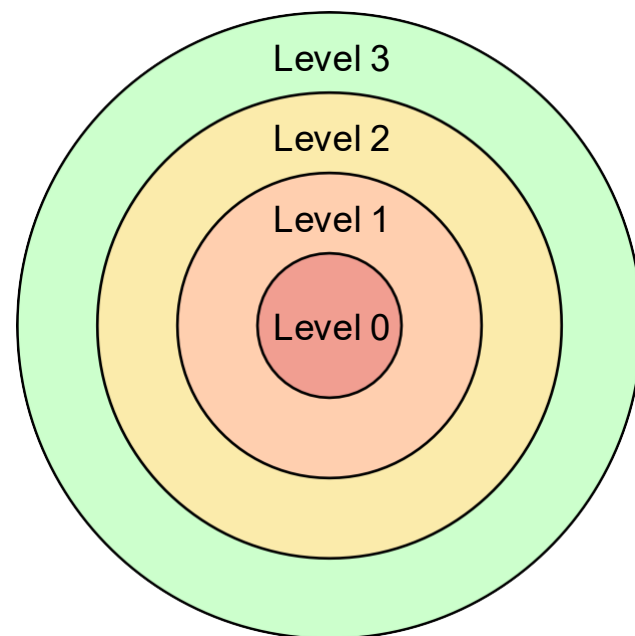
Multiple processor operating modes

■ Recent processors provide more than two operating modes

- Called hierarchical protection domains
- Higher-privilege rings can also access lower-privilege operations and data

■ Intel x86 provides four privilege rings

- Level 0 = kernel mode; Level 3 = user mode
- Modern OSes typically use only rings 0 and 3



■ Support for multiple protection levels is ubiquitous, even in mobile devices

- ARM processors support multiple privilege levels (exception levels), enabling fine-grained isolation

System calls

■ We saw that:

- Most software runs as user-level processes
- OS kernel runs in privileged mode

■ Therefore, when a user application requests a service from the OS, the system must transition from user to kernel mode to fulfill the request

■ Applications can invoke the kernel through **system calls**

- A special hardware instruction transfers control to kernel...
- ...which dispatches to one of few hundred system call handlers

System calls

- **Even simple applications rely heavily on OS services, since many essential operations cannot be performed in unprivileged (user) mode**
 - File system operations (read and write files)
 - Access to time and timers
 - Memory allocation and address space management
 - Process creation and management
 - ...
- **System calls allow applications to request privileged operations safely and in a controlled manner**

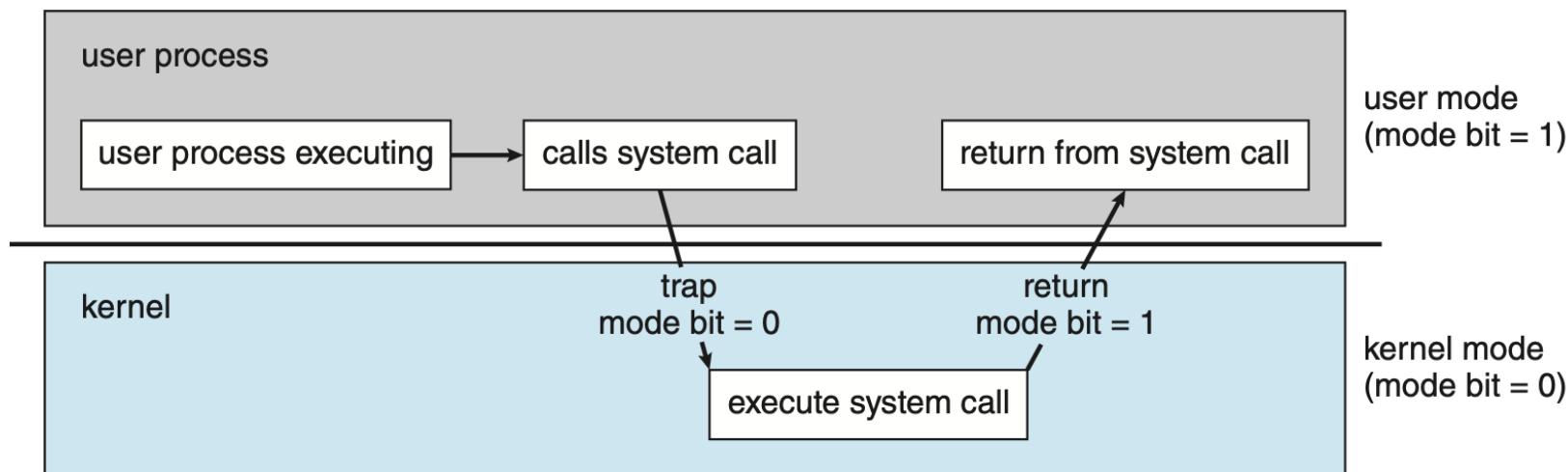
Operating modes and system calls

■ How does a system call interact with the OS?

- It issues a **trap instruction**

■ A trap is an intentional software-generated exception

- Also called **synchronous interrupts**, as they are produced by the CPU while executing instructions (more on the topic in a future lecture)



System calls

- **Typically, a number is associated with each system call**
 - The system call interface maintains a table indexed according to these numbers
 - The OS invokes the intended system call and returns status of the system call and any return values

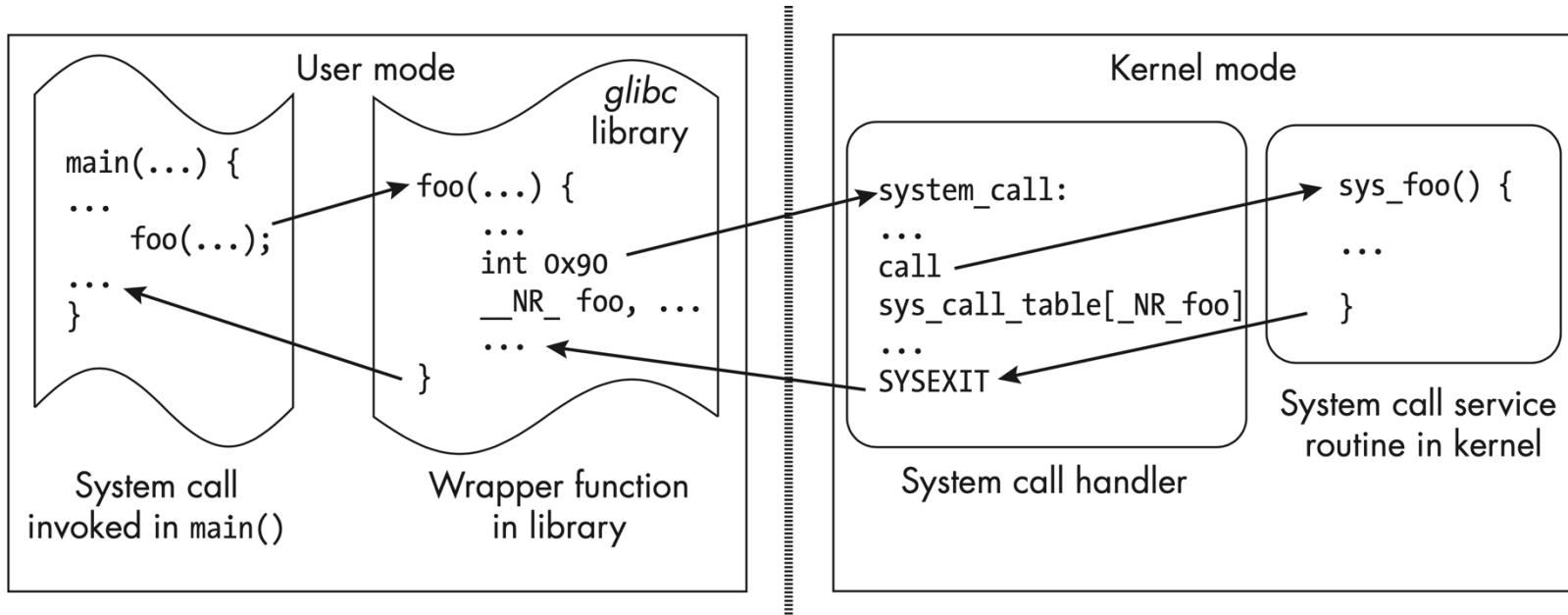
```
// open.S
.global open
open:
    mov $SYS_open, %eax
    syscall
    cmp $SYS_error_thresh, %rax
    ja __syscall_error
    ret
```

- **OS verifies applications' requests to prevent misbehavior**
 - E.g., check arguments to system calls, verify process permissions, etc.

Wrapper functions

- **Processes don't usually invoke system calls directly. Instead, they call wrapper functions**
 - The GNU C Library (`glibc`) has wrapper functions for almost all system calls
- **Wrapper functions for system calls usually have the same name as the call itself**
 - They also must execute the trap instruction to trap to kernel mode
- **The GNU C Library wrapper functions are often very thin**
 - Doing little more than copying arguments to the right registers before invoking the system call and then setting the value of a global error variable

Wrapper functions



Wrapper functions

■ You will use them several times during the semester

- This model is universal, in that these same four system calls (`open`, `read`, `write`, `close`) can be applied to all nondirectory files, including device files

```
#include <fcntl.h>
#include <unistd.h>

int main(){
    int fd = open("scomp.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if(fd < 0){
        write(2, "Failed to open scomp.txt\n", 25);
        exit(1);
    }

    write(fd, "Hello, OS!\n", 11);
    close(fd);
    return 0;
}
```

Libraries and system calls

- **Many functions in the C standard library act as user-space wrappers around kernel system calls**
 - Translating simple function calls into controlled transitions to kernel mode

```
FILE *fopen(const char *fname, const char *mode) {
    int flags = mode2flags(mode);

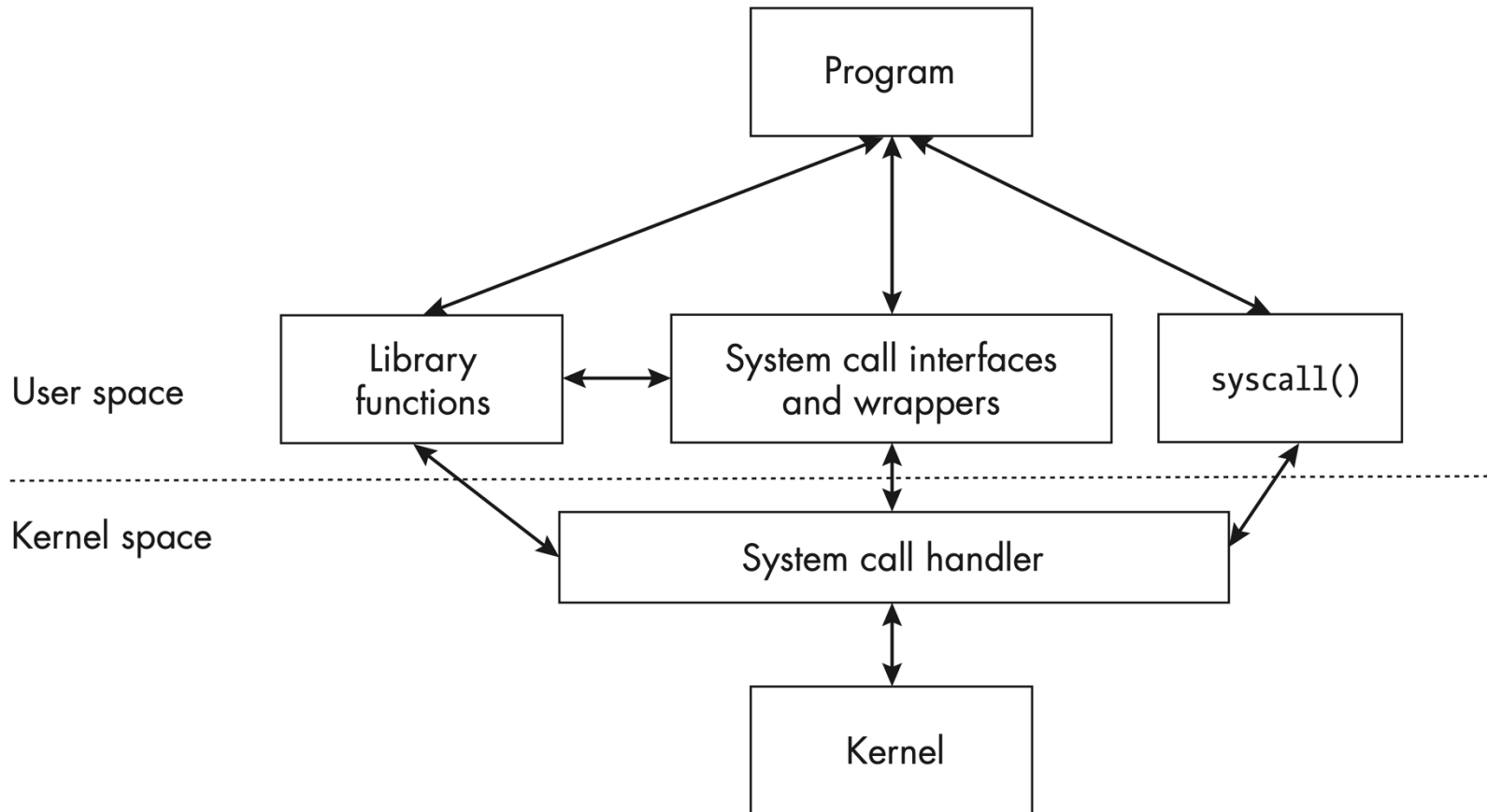
    if (!flags)
        return NULL;

    int fd = open(fname, flags, DEFPERMS);

    if (fd == -1)
        return NULL;

    return fdopen(fd, mode);
}
```

Multiple paths to kernel services



Tracing system calls

- **Typically, application developers design programs according to an application programming interface (API)**
 - Behind the scenes, the functions that make up an API typically invoke the actual system calls on behalf of the application programmer
- **The `strace` command line tool logs all system calls issued by an application and makes this information available to programmers**
 - Naturally, the kernel must provide special support for logging system calls
 - Not surprisingly, support is also provided in the form of a system call (`ptrace`)

System call error handling

■ Almost all system-level operations can fail

- Only exception is the handful of functions that return `void`
- A robust program should check the return value of system calls and handle every possible error

■ On error, most system-level functions return `-1` and set global variable `errno` to indicate cause

- `errno` is thread-local in modern systems

```
pid_t pid = fork();  
  
if (pid == -1) {  
    perror("fork error");  
    exit(1);  
}
```

Why applications are OS specific?

- **Applications compiled for one OS are usually not executable on another**
 - Each OS defines its own system call interface, its own binary formats (e.g., ELF, Mach-O), OS-specific conventions and runtime behavior
- **The Application Binary Interface (ABI) is the binary-level contract between an application and the OS on a given architecture**
 - It defines calling conventions, system call interface, binary format, register and stack usage
- **The ABI plays, at the binary level, a role similar to an API at the source-code level**

Types of architectural support

■ Manipulating privileged machine state

- Protected instructions – a subset of instructions of every CPU is restricted to be used only by the OS
- E.g., manipulate device registers, TLB entries, etc.

■ Generating and handling “events”

- System needs mechanisms for “exceptional control flow”
- Interrupts, exceptions, system calls, etc.

■ Memory protection

- OS must be able to protect programs from each other
- OS must protect itself from user programs

Hardware interrupts

- **Program control interruption based on a hardware event (external to the CPU)**
 - Generally, have nothing to do with the instructions currently executing
- **Some event informs the CPU that a device needs some attention**
 - The CPU interrupts the currently executing program, services the device, and then returns control back to the program
- **Also called asynchronous interrupts**
 - Generated by other hardware devices at arbitrary times with respect to the CPU clock signals

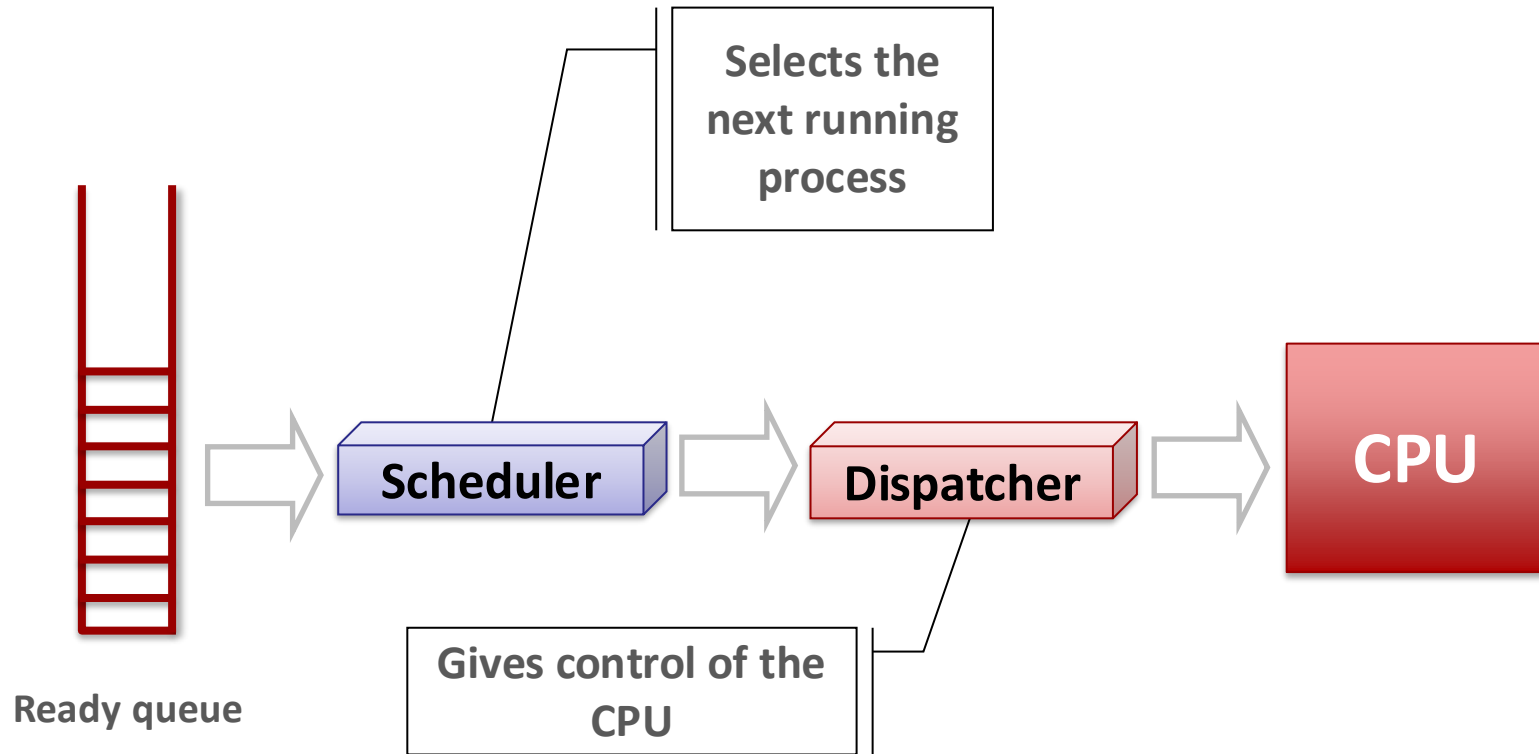
CPU protection

- **Early multitasking OSes provided cooperative multitasking**
 - Each process voluntarily relinquishes the CPU – e.g., when it waits for an I/O operation to complete, when it yields, or terminates
 - This would implicitly occur when certain system calls were made
- **With cooperative multitasking the OS is susceptible to badly designed programs that never relinquish the CPU**
 - E.g., buggy or malicious program
- **Therefore, it is necessary to ensure that the OS maintains control of the CPU**

Multitasking and the hardware timer

- Solution is to incorporate a **hardware timer** into the computer
- OS configures the timer to trigger after a specific amount of time
 - When time runs out, the hardware timer fires an interrupt
 - On Linux systems, the kernel configuration parameter `HZ` specifies the frequency of timer interrupts
- OS handles the interrupt by:
 1. Deciding which process should run next (**scheduling**)
 2. Performing a context-switch to another process, then starting the timer again (**dispatching**)

Scheduler and Dispatcher



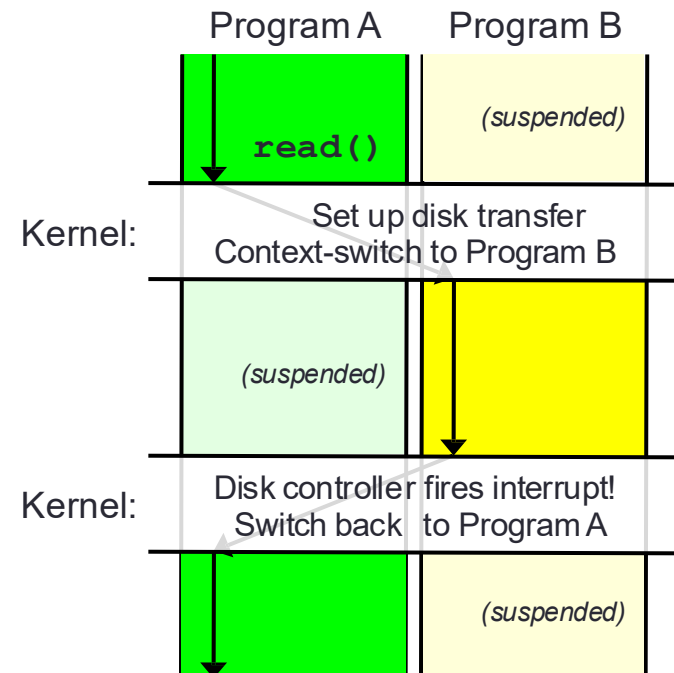
- Scheduler = policy
- Dispatcher = mechanism

Multitasking and the hardware timer

- An operating system provides **preemptive multitasking** when it can interrupt a currently running process and regain control of the CPU
 - The running process does not need to voluntarily yield
 - The OS decides when a context switch occurs
- All hardware timer configuration must require kernel-mode access
 - Without privileged timer control, fair scheduling would be impossible
- Virtually all widely used modern OSes implement preemptive multitasking

Long running I/O operations

- Hardware interrupt handling is also very important with long running I/O operations (and most of them are)
 - e.g., reading a block from a magnetic disk can take 3-15ms
 - e.g., reading a block from an SSD can take several μ s
- When a process performs a long running I/O operation:
 - The process is unable to proceed until the I/O operation completes
 - OS sets up the I/O operation, then **switches to a process that can run**
 - When the I/O operation is finished, the hardware fires an interrupt to notify the OS



Types of architectural support

■ Manipulating privileged machine state

- Protected instructions – a subset of instructions of every CPU is restricted to be used only by the OS
- E.g., manipulate device registers, TLB entries, etc.

■ Generating and handling “events”

- System needs mechanisms for “exceptional control flow”
- Interrupts, exceptions, system calls, etc.

■ Memory protection

- OS must be able to protect programs from each other
- OS must protect itself from user programs

Why memory protection?

■ Multiple processes execute concurrently

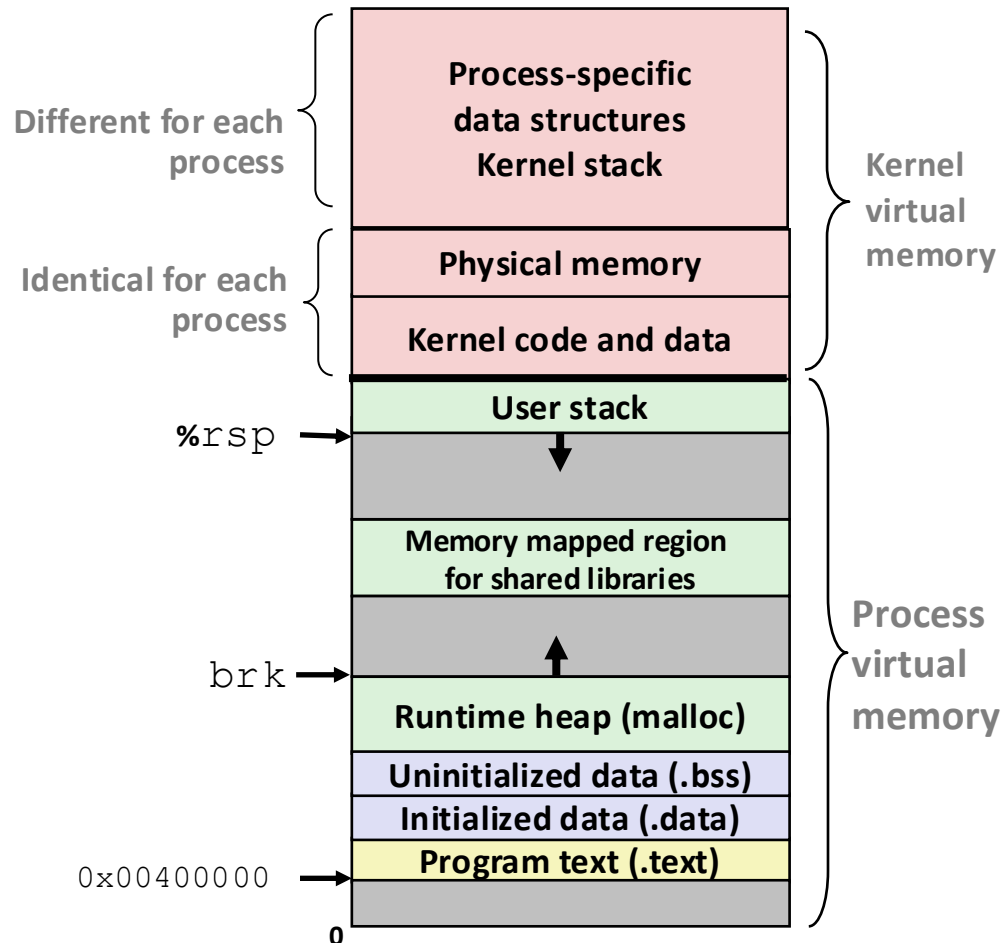
- Each process must be isolated
- A faulty process must not corrupt the kernel, other processes, or the system as a whole

■ Without memory protection

- Bugs become system crashes
- One process can overwrite another
- Security becomes impossible

Virtual memory

- Virtual memory allows regions of memory to be restricted
 - To kernel-mode access only, or allow user-mode access



Virtual memory

- Each process runs in its own virtual address space, defined by a page table managed by the OS
- Two processes may use the same virtual address while referring to entirely different physical memory
 - This ensures that concurrent processes are isolated by construction
 - Memory races between processes are impossible by default, unless the OS explicitly allows sharing
- VM therefore acts as the **first and strongest concurrency control mechanism** in the system

Virtual memory

■ Memory protection relies fundamentally on hardware support

- The Memory Management Unit (MMU) enforces address translation and access permissions
- The Translation Lookaside Buffer (TLB) accelerates address translation

■ Without dedicated hardware mechanisms

- Implementing virtual memory and protection purely in software would introduce prohibitive performance overhead

■ When a process violates protection:

1. The CPU raises an exception
 2. The OS handles the fault
- (a topic for a future lecture)

Summary

■ Key roles of an OS

- Allocates CPU, memory, and I/O resources efficiently
- Supports multitasking and multithreading for simultaneous task execution
- Provides simplified interfaces for hardware resources through APIs

■ Why it matters

- Ensures system stability and performance
- Facilitates software development by abstracting hardware complexities

Summary

- **What an OS can do is dictated, at least in part, by the architecture**
 - Architecture support can greatly simplify (or complicate) OS tasks
- **Modern OSes rely heavily on hardware facilities to provide their functionality**
 - At least dual-mode execution, kernel mode and user mode
 - Ability to trap (or otherwise transfer control) into kernel-mode code
 - Support for both software (e.g., traps, faults) and hardware interrupts (e.g., I/O devices)
 - A hardware timer facility to allow OS to retain control of the CPU
 - Virtual memory management, including memory protection to enforce barrier between kernel space and user space